OpenFabric: From Distributed Tensor Placement to Executable Manycore Tensor Accelerator Packages

Anonymous Authors
Anonymous Institution
Anonymous City, Country
anonymous@example.com

***Abstract*—Emerging tensor accelerators often expose manycore execution fabrics, explicit local memories, vendor-specific task formats, and closed runtime package interfaces. These machines can deliver high throughput, but deploying new model operators still depends heavily on case-by-case expert kernels and brittle template code. We present OpenFabric, a distributed tensor compilation substrate that lowers DTensor-style placement semantics into per-core tile programs and executable backend packages. OpenFabric makes logical collectives first-class: data visibility, broadcast, reduction, and materialization obligations are represented explicitly before they are expanded into backend graph nodes, task/subtask schedules, instance tables, base-address records, and binary runtime package files. Its central IR, TileScope/Value/View, separates scheduling ownership from tensor-core internal state, allowing accumulators, temporary fragments, materialized tiles, and fused post-ops to be validated at the same abstraction level. On a DFU/GPDPU manycore tensor accelerator, OpenFabric generates executable vendor runtime packages from high-level operator programs and recovers approximately 70% of expert-tuned kernel throughput on evaluated operators, while preserving an inspectable lowering path that replaces hand-maintained template flows.**

***Index Terms*—tensor compiler, distributed tensor, manycore accelerator, collective communication, operator generation, runtime package**

I. Introduction

The programming gap for tensor accelerators is no longer limited to writing a single fast kernel. Practical deployment requires a repeatable path from model operators to target-specific execution packages across changing shapes, fused operators, memory layouts, and hardware generations. This gap is especially visible on emerging accelerators: the hardware may expose a useful mesh of tensor cores, local memories, and data movement engines, but the software path often remains a collection of vendor examples, handwritten C/C++ templates, CSV-like instruction streams, task descriptors, base-address tables, and closed runtime package formats.

Mainstream systems have strong compiler ecosystems for tensor kernels, yet their abstractions do not directly solve this problem. Triton gives programmers a productive tile-kernel language for GPUs [1]. Distributed tensor systems expose placement concepts such as shard, replicate, and partial reduction [2]. Recent systems push tile programs toward spatial dataflow accelerators [3] or extend Triton with distributed communication overlap [4]. However, a deployment compiler for emerging tensor fabrics must connect all of these concerns at once: it must preserve distributed tensor semantics, expose collective visibility obligations, lower through tile-level schedules, and finally materialize the exact backend runtime package consumed by the target system.

OpenFabric is our answer to this problem. It treats a manycore accelerator as a small distributed tensor machine. A user describes an operator using DTensor-like tensor placement and ordinary compute operations. The compiler derives logical collective obligations, assigns tile ownership to tensor cores, builds per-core tile programs, and lowers those programs into backend execution artifacts. The first backend is DFU/GPDPU, a constrained 4×4 tensor-core mesh with an explicit task/subtask/instance runtime model and a vendor package ABI. This backend is intentionally demanding: it forces the compiler to account for tile residency, tensor-core temporary state, instruction payloads, shared instance tables, base-address slots, and final binary package composition.

This paper makes the following contributions:

- We introduce a placement-first compiler architecture that lowers distributed tensor semantics into per-core tile programs, rather than starting from a handwritten tile kernel.
- We make logical collectives and visibility obligations first-class IR objects, allowing broadcasts, reductions, materialization, and stores to be validated before backend expansion.
- We present TileScope/Value/View, an IR split that models both schedulable tile ownership and tensor-core internal state such as accumulator fragments, summary values, and materialized tile views.
- We implement a DFU/GPDPU backend that emits executable vendor runtime packages, including task/subtask schedules, exeBlock descriptors, shared instance rows, base-address records, instruction blobs, and package files.
- We evaluate generated GEMM, fused GEMM, and non-GEMM collective operators, showing that OpenFabric recovers roughly 70% of expert-written kernel performance while replacing case-specific template maintenance with an inspectable compiler flow.

II. Motivation and Backend Constraints

A. From Operator Semantics to Runtime Packages

The deployment workflow that motivated OpenFabric starts from model operators, but the backend consumes low-level

runtime package files. A typical manual path interleaves semantic decisions and hardware details:

> operator shape and layout → PE map → tile loops → template C/C++ → CSV/instruction records → task/subtask/instance tables → runtime package

This path is fragile because each new operator reopens the same questions: which tensor dimensions are sharded, which values must become visible to which PEs, where reductions occur, when a fused post-op can stay local, and which task descriptor or base-address slot carries each piece of state.

B. DFU/GPDPU as a Stress Test

The DFU/GPDPU backend exposes a 4×4 PE mesh. Each PE executes local tensor instructions and participates in explicit data movement. The runtime package contains task and subtask metadata, PE-specific execution blocks, shared instance tables, instruction payload blobs, and input/output data files. Memory references are partly relocatable: instruction offsets are combined with per-instance base-address slots. These constraints are not incidental engineering details; they shape the compiler IR. If the compiler loses the identity of a tile, a collective obligation, or a tensor-core temporary value, it cannot later prove that a generated runtime package is correct.

III. OpenFabric Overview

Fig. 1 shows the OpenFabric pipeline. The frontend borrows the useful static subset of distributed tensor programming: a device mesh, logical tensor metadata, placements, local compute operations, partial results, and explicit redistribution intent. The backend is responsible for target expansion: tile ownership, tile actions, collective routing, graph nodes, runtime frames, binary records, and final package files.

The central design rule is:

> The tile program is the program; dependency graphs and vendor package records are derived views.

This rule prevents backend artifacts from becoming the only source of truth. Debug and validation views can be regenerated from structured IR, while binary package emission remains a final materialization step.

IV. Semantic Model

A. Placement-First Operators

OpenFabric begins with logical tensors over a mesh. Each tensor carries shape, dtype, and placement metadata. Placements include sharding over a mesh dimension, replication, and partial values that require a later reduction. For example, matrix multiplication on a 4×4 mesh can be expressed as row-sharded output tiles, row-visible tiles of A, and column-visible tiles of B. The source program expresses what tensors mean; it does not manually enumerate backend task files.

This choice is intentionally different from kernel-first DSLs. A kernel-first program tells one program instance how to load, compute, and store a tile. OpenFabric first asks which distributed tensor values must be visible to each core, then derives local tile programs and backend actions.

B. LogicalCollective

A LogicalCollective records a visibility or reduction obligation before physical routing is chosen. It includes the source logical value, participant group, visibility kind, role information, and consumer tile actions. For a SUMMA-style GEMM, one A tile becomes row-visible and one B tile becomes column-visible at each K step. For a signal-processing operator such as `log10 -> max -> maximum`, local logarithms and local maxima are computed first, a global max reduction is represented as a logical all-reduce, and the post-reduction maximum and affine scale remain PE-local.

LogicalCollectives are not backend graph nodes. They are semantic obligations that later lower into per-participant TileCollectiveActions, route edges, instruction payloads, or runtime records depending on the backend.

V. TileScope/Value/View IR

A. TileScope

A TileScope is a schedulable ownership domain. It records a logical tile identity, owner core, global range, local actions, collective participation, member values, materialization state, and backend schedule hints. TileScope is the unit that users and compiler developers inspect when deciding whether an operator has been partitioned correctly.

B. Value and View

Many tensor-core instructions do not directly produce a fully materialized output tile. For example, a matrix multiply instruction may accumulate into temporary tensor state, while later import/export instructions move between temporary state and ordinary operand storage. Therefore OpenFabric separates:

- *Value*: the actual producer/consumer object, such as an operand strip, accumulator fragment, local summary scalar, or temporary tensor state.
- *View*: a structured interpretation of one or more values as a tile-shaped semantic object, such as an accumulator tile or fused post-op input.

This split lets the compiler keep tile-level scheduling visible while still respecting instruction-level producer/consumer boundaries. It also supports fusion: a ReLU, maximum, or affine scale can consume a tile view and materialize only the final output tile.

C. Tile Actions

Each TileScope contains a timeline of actions:

- TileCollectiveAction for visibility, broadcast, reduction, and store obligations.
- ComputeTileAction for tensor-core operations, element-wise operations, and local reductions.
- ViewAction for constructing semantic tile views over member values.
- MaterializeTileAction for exposing a view to memory or to later backend records.
- BarrierAction for phase boundaries that cannot be fused.

Operator Program → DTensor-style placement propagation → chip logical actions and LogicalCollectives → per-core Tile Programs with TileScope/Value/View → Global Tile Dependency Network → backend graph and packing → task/subtask/exeBlock/instance/base-table records → binary vendor runtime package

Fig. 1. OpenFabric lowers distributed tensor placement into backend runtime packages through explicit logical collectives and per-core tile programs.

TABLE I
DFU/GPDPU LOWERING STAGES

Stage	Compiler responsibility
Tile program	Per-PE TileScopes, K steps, actions, value/view state
Dependency network	Derived tile values, dependencies, collective consumers
Backend graph	PE-local graph nodes and cross-PE route edges
Packing	Task/subtask/instance grouping and resource checks
Runtime frame	Symbolic buffers, workspace, IO binding, base symbols
Vendor ABI	exeBlock rows, instance rows, task/subtask records
Binary package	Encoded blobs and `config/*.bin` package files

This action model is the reason the same IR can represent GEMM and non-GEMM collective operators. GEMM specializes the model with K-streaming and row/column visibility. Other operators use the same local/collective/post-local structure.

VI. Backend Lowering to Executable Packages

A. Lowering Stages

The DFU/GPDPU backend lowers tile programs through the stages in Table I. The early stages preserve semantic structure; the late stages conform to the vendor ABI.

B. Vendor Runtime Package

The generated package contains the files expected by the closed runtime: `cbuf_file.bin`, `micc_file.bin`, `input_data.bin`, and `riscv_program`. The compiler constructs the first two from lower-level surfaces:

- instruction records and exeBlock/instance descriptors for the CBUF blob;
- task and subtask descriptors for the MICC blob.

The same structured plan also records provenance: every binary record can be traced back to a TileScope, logical collective, tile dependency, or runtime frame symbol.

C. Relocatable Materialization

OpenFabric supports a practical form of relocatable kernel generation. The compiled instruction template is fixed for an operator shape and tiling, while the materialization context binds logical input, output, and workspace buffers to concrete base-address slots. Effective addresses are computed as:

$$\textit{effective\_address} = \textit{instance.base}[\textit{base\_addr\_idx}] + \textit{offset}. \quad (1)$$

This separates operator code generation from deployment-time memory placement and allows the same generated kernel template to be materialized for different runtime buffers.

VII. Implementation

The current implementation is written in Python and emits both structured JSON plans and human-readable debug views. The debug views are not build inputs; they are review surfaces for developers and for validation. For the GEMM+ReLU baseline, the compiler emits per-PE tile phases, derived collective obligations, route summaries, task plans, structured assembly records, tile dependency networks, graph packing plans, runtime frames, base-table rows, vendor package layout views, exeBlock rows, shared instance rows, and final binary package files.

The validator checks structural invariants across these layers:

- every TileCollectiveAction is backed by a LogicalCollective;
- every consumer-visible tile has a producer or materialization path;
- task/subtask/instance packing respects backend capacity constraints;
- base-address slots are consistently assigned across shared instance rows;
- binary record counts match the package manifest and runtime ABI.

VIII. Evaluation

We evaluate OpenFabric along four dimensions: correctness, performance, generated artifact structure, and programmability. All results use the DFU/GPDPU 4×4 tensor-core mesh.

A. Correctness

Generated kernels are compared against reference implementations for every evaluated operator. For floating-point outputs, we use standard relative and absolute tolerance checks. We additionally validate the generated package structure before runtime execution, ensuring that package records are consistent with the compiler plan.

B. Performance

Table II summarizes throughput relative to expert-written vendor kernels. OpenFabric does not aim to beat expert micro-tuned kernels. Its goal is to recover most of the bulk performance by preserving tile locality, data reuse, PE parallelism, explicit collective boundaries, and fused post-ops, while substantially reducing manual template work.

TABLE II
PERFORMANCE RELATIVE TO EXPERT-WRITTEN KERNELS

Operator	Naive	OpenFabric	Expert
GEMM	0.41×	0.72×	1.00×
GEMM+ReLU	0.39×	0.70×	1.00×
log10-max-maximum	0.36×	0.68×	1.00×
Conv2d lowering	0.38×	0.69×	1.00×
Geomean	0.38×	0.70×	1.00×

TABLE III
GENERATED ARTIFACT STRUCTURE FOR GEMM+RELU

Artifact	Count
PE tile programs	16
Local GEMM phases per PE	4
K-block updates per phase	4
Logical collective obligations	128
Vendor tasks	4
Vendor active subtasks per task	3
Vendor exeBlocks	256
Shared vendor instance rows	24

C. Generated Structure

Table III shows the generated structure for the GEMM+ReLU baseline. These counts are useful because the backend ABI is not a single flat kernel: performance and correctness depend on how semantic tile work is folded into tasks, subtasks, exeBlocks, and shared instance rows.

D. Programmability and Inspectability

The manual baseline requires developers to maintain operator-specific PE maps, tile loops, instruction templates, runtime tables, and package assembly scripts. OpenFabric moves these concerns into compiler passes. Developers inspect the generated plan at the level where each bug naturally appears: placements and logical actions for semantic errors, TileScopes for tiling errors, collective obligations for visibility errors, and package records for backend ABI errors.

IX. DISCUSSION

A. Why Performance Is Plausible Without Expert Micro-Tuning

Expert kernels still have advantages: instruction scheduling, bank-conflict avoidance, backend-specific template shrinking, and route-level micro-tuning. OpenFabric targets the larger structural wins first. Its scheduler keeps tile-local producer/consumer chains together, reuses row/column-visible tiles, keeps accumulators resident through TileScope values and views, cuts phases only at collective or barrier boundaries, and fuses post-ops before stores. These principles capture the main data-reuse and parallelism structure of expert kernels while leaving backend micro-optimization as an orthogonal pass.

B. Scope

OpenFabric is not a replacement for a full model framework. It is a compiler substrate for operator generation and backend package materialization. It also does not claim that every backend should expose DFU-like task files. Rather, DFU/GPDPU demonstrates that the same placement/collective/tile-program abstractions can survive contact with a highly constrained legacy runtime.

X. RELATED WORK

Triton. Triton introduced a productive tile-centered language and compiler for neural network computations on GPUs [1]. OpenFabric shares the goal of replacing low-level handwritten kernels with compiler generated code, but starts at a different abstraction level. Triton is kernel-first: programmers define tile/block programs. OpenFabric is placement-first: the compiler derives per-core tile programs from distributed tensor semantics and logical collectives.

Tile-based spatial dataflow compilers. TileLoom compiles tile-based programs, including Triton kernels, onto spatial dataflow accelerators and plans dataflow across spatially distributed cores [3]. OpenFabric is complementary: it begins from DTensor-style placement and collective intent, then lowers to backend package records. The key distinction is not whether both systems care about tiles, but where tile programs come from and how collective visibility obligations are represented.

Distributed Triton extensions. Triton-distributed extends Triton with communication primitives and compiler-assisted overlapping for distributed AI systems [4]. OpenFabric instead targets chip-internal manycore tensor fabrics and package-level deployment. Its collective actions are not only performance overlap opportunities; they are semantic obligations that must be reflected in tile programs, route plans, and backend runtime records.

Multi-level GPU lowering. ML-Triton argues that direct lowering from a workgroup to per-thread representation is premature and introduces a multi-level flow aligned with GPU hierarchy [5]. OpenFabric similarly uses gradual lowering, but its hierarchy is distributed-tensor placement to chip-level logical collective to tile action to backend package.

Distributed tensors. DTensor-style APIs provide a powerful vocabulary for mesh placement, sharding, replication, and partial reductions [2]. OpenFabric uses this vocabulary as a static compiler frontend, not as a runtime execution system. The backend-specific contribution is the lowering of that vocabulary into concrete tile schedules and executable manycore packages.

XI. CONCLUSION

OpenFabric turns distributed tensor placement into executable manycore tensor accelerator packages. By making logical collectives first-class and separating TileScope ownership from Value/View state, it preserves the information needed for both compiler validation and backend materialization. The DFU/GPDPU case study shows that this abstraction can bridge

from high-level operator programs to a constrained vendor runtime ABI while recovering a substantial fraction of expert kernel performance. More broadly, OpenFabric suggests that placement-aware, collective-aware tile programs are a useful substrate for both emerging accelerators and future backend-specific tensor execution fabrics.

REFERENCES

[1] P. Tillet, H. T. Kung, and D. Cox, "Triton: An intermediate language and compiler for tiled neural network computations," in *Proc. MAPL*, 2019.

[2] PyTorch Contributors, "Distributed tensor," PyTorch documentation. [Online]. Available: https://pytorch.org/docs/stable/distributed.tensor.html

[3] W. Li, Z. Bai, H. Wang, P. Dangi, Z. Zhang, C. Tan, H. Lan, W.-F. Wong, and T. Mitra, "TL: Automatic end-to-end compiler of tile-based languages for spatial dataflow architectures," arXiv:2512.22168, 2025.

[4] S. Zheng et al., "Triton-distributed: Programming overlapping kernels on distributed AI systems with the Triton compiler," arXiv:2504.19442, 2025.

[5] D. Wang, W. Zhu, L. Ling, E. Tiotto, Q. Wang, W. Tsang, J. Opperman, and J. Deng, "ML-Triton, a multi-level compilation and language extension to Triton GPU programming," arXiv:2503.14985, 2025.